

Top 3 Matplotlib Plotting Styles That Make Your Plots Look Professional

Transform Ugly Plots into Publication-Ready Visualizations

Video Title: *3 Matplotlib Styles That Make Your Plots Look Professional*

Why These Styles?

- Default matplotlib looks amateur and dated
- Professional publications require clean, modern aesthetics
- One line of code can transform your entire visualization

What You'll Learn:

1. **Seaborn Style** - Clean, modern, research-ready
2. **Dark Background** - Perfect for presentations and screens
3. **BMH (Bayesian Methods for Hackers)** - Publication-quality aesthetics
4. Bonus: Custom style creation and quick switches

The Problem:

- Default matplotlib plots look like they're from 2003
- Colors clash, gridlines are harsh, fonts are boring
- Your analysis is great, but plots don't do it justice

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib import style
import seaborn as sns

# Generate sample data
np.random.seed(42)
x = np.linspace(0, 10, 100)
y1 = np.sin(x) + np.random.normal(0, 0.1, 100)
y2 = np.cos(x) + np.random.normal(0, 0.1, 100)
y3 = np.sin(x) * np.cos(x) + np.random.normal(0, 0.1, 100)

# DataFrame for more complex plots
df = pd.DataFrame({
    'Category': ['A', 'B', 'C', 'D', 'E'] * 20,
    'Value': np.random.randn(100) * 10 + 50,
    'Size': np.random.randint(10, 100, 100)
})
```

0. The Problem: Default Style (Before)

```
In [2]: print("PROBLEM: DEFAULT MATPLOTLIB STYLE")
print("="*70)

# Reset to default
plt.style.use('default')

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Line plot
axes[0].plot(x, y1, label='Sin wave')
axes[0].plot(x, y2, label='Cos wave')
axes[0].set_title('Line Plot (Default)')
axes[0].legend()
axes[0].grid(True)

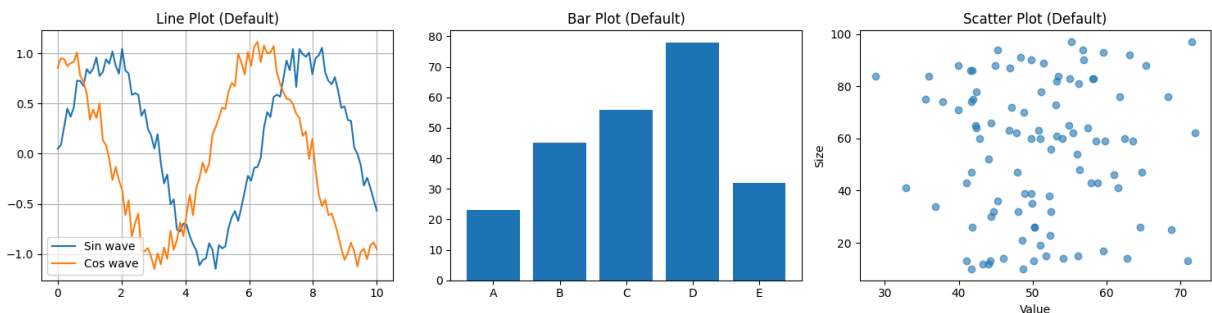
# Bar plot
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 78, 32]
axes[1].bar(categories, values)
axes[1].set_title('Bar Plot (Default)')

# Scatter plot
axes[2].scatter(df['Value'], df['Size'], alpha=0.6)
axes[2].set_title('Scatter Plot (Default)')
axes[2].set_xlabel('Value')
axes[2].set_ylabel('Size')

plt.tight_layout()
plt.show()

print("\n❌ PROBLEMS WITH DEFAULT:")
print("    • Harsh colors (pure blue #0000FF)")
print("    • Thick, dark gridlines that overpower data")
print("    • Boring, small fonts")
print("    • No color palette consistency")
print("    • Looks dated and unprofessional")
print("\n💡 Solution: Use professional styles!")
print("="*70)
```

PROBLEM: DEFAULT MATPLOTLIB STYLE



- ❌ PROBLEMS WITH DEFAULT:
- Harsh colors (pure blue #0000FF)
 - Thick, dark gridlines that overpower data
 - Boring, small fonts
 - No color palette consistency
 - Looks dated and unprofessional

💡 Solution: Use professional styles!

=====

1. Style #1: Seaborn - Modern & Clean (Research/Analysis)

```
In [3]: print("\nSTYLE #1: SEABORN - MODERN & CLEAN")
print("="*70)

# Apply seaborn style
plt.style.use('seaborn-v0_8')

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Line plot
axes[0].plot(x, y1, label='Sin wave', linewidth=2)
axes[0].plot(x, y2, label='Cos wave', linewidth=2)
axes[0].set_title('Line Plot (Seaborn)', fontsize=14, fontweight='bold')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Bar plot
axes[1].bar(categories, values)
axes[1].set_title('Bar Plot (Seaborn)', fontsize=14, fontweight='bold')

# Scatter plot
axes[2].scatter(df['Value'], df['Size'], alpha=0.6, s=50)
axes[2].set_title('Scatter Plot (Seaborn)', fontsize=14, fontweight='bold')
axes[2].set_xlabel('Value')
axes[2].set_ylabel('Size')

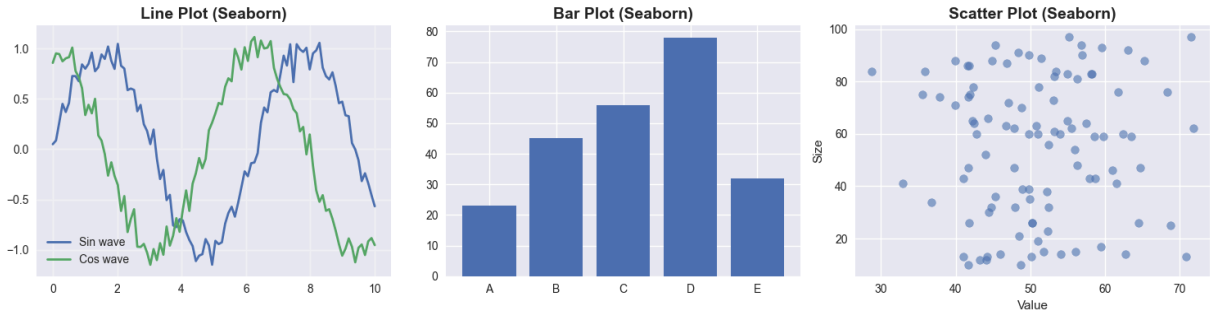
plt.tight_layout()
plt.show()

print("\n✅ SEABORN STYLE FEATURES:")
print("    • Muted, pastel colors (easy on eyes)")
print("    • Light gridlines with gray background")
print("    • Better default fonts and sizes")
print("    • White axes spines (cleaner look)")
print("    • Professional color palette")

print("\n🌟 WHEN TO USE:")
print("    • Research papers and reports")
print("    • Data analysis notebooks")
print("    • When you need clean, readable plots")
print("    • Jupyter notebooks and documentation")
```

```
print("\n📄 CODE:")
print("    plt.style.use('seaborn-v0_8') # One line!")
print("    # Or use context manager:")
print("    with plt.style.context('seaborn-v0_8'):")
print("        plt.plot(x, y) # Only affects this plot")
print("====*70")
```

STYLE #1: SEABORN – MODERN & CLEAN



✓ SEABORN STYLE FEATURES:

- Muted, pastel colors (easy on eyes)
- Light gridlines with gray background
- Better default fonts and sizes
- White axes spines (cleaner look)
- Professional color palette

🎯 WHEN TO USE:

- Research papers and reports
- Data analysis notebooks
- When you need clean, readable plots
- Jupyter notebooks and documentation

📄 CODE:

```
plt.style.use('seaborn-v0_8') # One line!
# Or use context manager:
with plt.style.context('seaborn-v0_8'):
    plt.plot(x, y) # Only affects this plot
```

2. Style #2: Dark Background - Presentations & Screens

```
In [4]: print("\nSTYLE #2: DARK BACKGROUND – PRESENTATIONS")
print("====*70")

# Apply dark background
plt.style.use('dark_background')

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.patch.set_facecolor('#0E1117') # Match dark theme

# Line plot with vibrant colors
axes[0].plot(x, y1, label='Sin wave', linewidth=2.5, color='#FF4B4B')
axes[0].plot(x, y2, label='Cos wave', linewidth=2.5, color='#00D9FF')
axes[0].set_title('Line Plot (Dark)', fontsize=14, fontweight='bold', color=
```

```

axes[0].legend()
axes[0].grid(True, alpha=0.2, color='gray')
axes[0].set_facecolor('#0E1117')

# Bar plot
bars = axes[1].bar(categories, values, color=['#FF4B4B', '#00D9FF', '#FFD700'])
axes[1].set_title('Bar Plot (Dark)', fontsize=14, fontweight='bold', color='white')
axes[1].set_facecolor('#0E1117')

# Scatter plot
scatter = axes[2].scatter(df['Value'], df['Size'], alpha=0.7, s=50, c=df['Size'])
axes[2].set_title('Scatter Plot (Dark)', fontsize=14, fontweight='bold', color='white')
axes[2].set_xlabel('Value', color='white')
axes[2].set_ylabel('Size', color='white')
axes[2].set_facecolor('#0E1117')
plt.colorbar(scatter, ax=axes[2])

plt.tight_layout()
plt.show()

print("\n✅ DARK BACKGROUND FEATURES:")
print("    • Black/dark gray background")
print("    • Vibrant, high-contrast colors")
print("    • White text and labels")
print("    • Reduced eye strain in dark rooms")
print("    • Modern, sleek appearance")

print("\n👁️ WHEN TO USE:")
print("    • Presentations in dark rooms")
print("    • Dashboards and monitoring screens")
print("    • Video content and streaming")
print("    • Dark-themed websites/apps")
print("    • Evening work (reduces eye strain)")

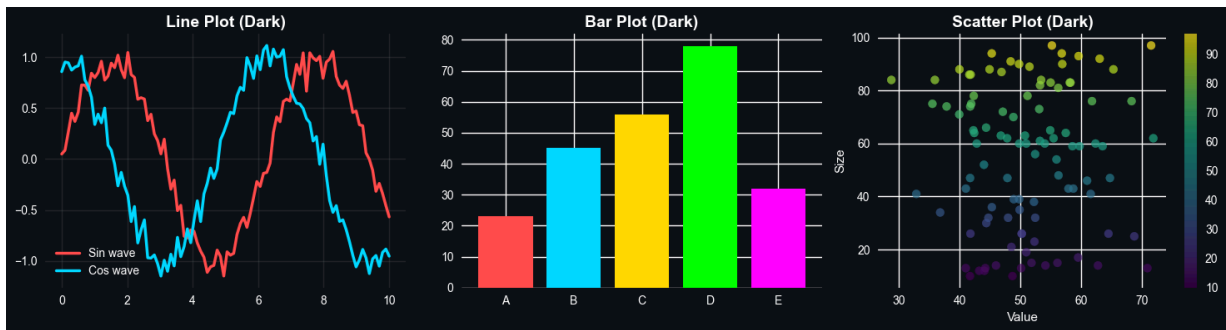
print("\n📄 CODE:")
print("    plt.style.use('dark_background')")
print("    # Pro tip: Match figure background")
print("    fig.patch.set_facecolor('#0E1117')")
print("    ax.set_facecolor('#0E1117')")

print("\n⚠️ PRO TIP:")
print("    Use HIGH-CONTRAST colors on dark backgrounds!")
print("    Good: #FF4B4B, #00D9FF, #FFD700")
print("    Bad: Dark blue, dark red on black")
print("\n*70")

```

STYLE #2: DARK BACKGROUND – PRESENTATIONS

=====



- ✓ DARK BACKGROUND FEATURES:
- Black/dark gray background
 - Vibrant, high-contrast colors
 - White text and labels
 - Reduced eye strain in dark rooms
 - Modern, sleek appearance

- 🎯 WHEN TO USE:
- Presentations in dark rooms
 - Dashboards and monitoring screens
 - Video content and streaming
 - Dark-themed websites/apps
 - Evening work (reduces eye strain)

📄 CODE:

```
plt.style.use('dark_background')
# Pro tip: Match figure background
fig.patch.set_facecolor('#0E1117')
ax.set_facecolor('#0E1117')
```

⚠️ PRO TIP:

Use HIGH-CONTRAST colors on dark backgrounds!

Good: #FF4B4B, #00D9FF, #FFD700

Bad: Dark blue, dark red on black

3. Style #3: BMH - Publication Quality

```
In [5]: print("\nSTYLE #3: BMH - PUBLICATION QUALITY")
print("="*70)

# Apply BMH style (Bayesian Methods for Hackers)
plt.style.use('bmh')

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Line plot
axes[0].plot(x, y1, label='Sin wave', linewidth=2.5)
axes[0].plot(x, y2, label='Cos wave', linewidth=2.5)
axes[0].plot(x, y3, label='SinxCos', linewidth=2.5)
axes[0].set_title('Line Plot (BMH)', fontsize=14, fontweight='bold')
axes[0].legend(frameon=True, shadow=True)
axes[0].grid(True, alpha=0.4)

# Bar plot
```

```

axes[1].bar(categories, values, edgecolor='white', linewidth=1.5)
axes[1].set_title('Bar Plot (BMH)', fontsize=14, fontweight='bold')

# Scatter plot
axes[2].scatter(df['Value'], df['Size'], alpha=0.6, s=50, edgecolors='white')
axes[2].set_title('Scatter Plot (BMH)', fontsize=14, fontweight='bold')
axes[2].set_xlabel('Value')
axes[2].set_ylabel('Size')

plt.tight_layout()
plt.show()

print("\n✅ BMH STYLE FEATURES:")
print("    • Subtle blue/tan background")
print("    • Bold, saturated colors")
print("    • Thick axes spines")
print("    • White edges on plot elements")
print("    • Named after famous Bayesian book")

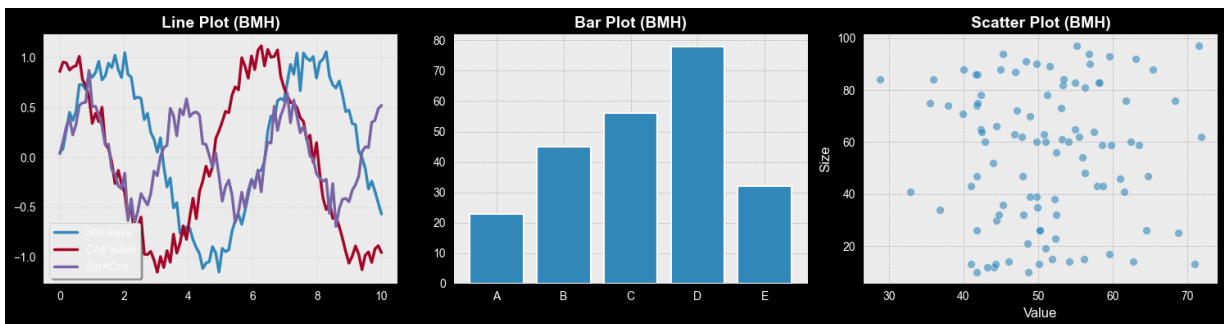
print("\n🎯 WHEN TO USE:")
print("    • Academic papers and journals")
print("    • Books and printed materials")
print("    • Formal presentations")
print("    • When you need 'serious' aesthetics")
print("    • Statistical analysis reports")

print("\n📄 CODE:")
print("    plt.style.use('bmh')")

print("\n📖 BACKGROUND:")
print("    BMH = 'Bayesian Methods for Hackers'")
print("    Named after Cameron Davidson-Pilon's book")
print("    Designed specifically for statistical plots")
print("\n="*70)

```

STYLE #3: BMH – PUBLICATION QUALITY



- ✅ **BMH STYLE FEATURES:**
 - Subtle blue/tan background
 - Bold, saturated colors
 - Thick axes spines
 - White edges on plot elements
 - Named after famous Bayesian book

- 🎯 **WHEN TO USE:**
 - Academic papers and journals
 - Books and printed materials
 - Formal presentations
 - When you need 'serious' aesthetics
 - Statistical analysis reports

💻 **CODE:**
`plt.style.use('bmh')`

📖 **BACKGROUND:**
BMH = 'Bayesian Methods for Hackers'
Named after Cameron Davidson-Pilon's book
Designed specifically for statistical plots

=====

4. Available Styles & How to List Them

```
In [6]: print("\nALL AVAILABLE MATPLOTLIB STYLES")
        print("=*70)

        # List all available styles
        available_styles = plt.style.available
        print(f"\nTotal styles available: {len(available_styles)}\n")

        print("📋 FULL LIST:")
        for i, style_name in enumerate(sorted(available_styles), 1):
            print(f"    {i:2d}. {style_name}")

        print("\n★ MOST POPULAR STYLES:")
        popular = ['seaborn-v0_8', 'ggplot', 'dark_background', 'bmh', 'fivethirtyeight',
                  'grayscale', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-whitegrid']

        for style_name in popular:
            if style_name in available_styles:
                print(f"    ✓ {style_name}")

        print("\n📖 HOW TO USE:")
        print("    # Global (affects all plots)")
        print("    plt.style.use('seaborn-v0_8')")
        print("")
        print("    # Temporary (only one plot)")
        print("    with plt.style.context('dark_background'):")
        print("        plt.plot(x, y)")
        print("")
        print("    # Reset to default")
```



```
print("    plt.style.use('default')")  
print("=*70)
```

ALL AVAILABLE MATPLOTLIB STYLES

=====

Total styles available: 29

FULL LIST:

1. Solarize_Light2
2. _classic_test_patch
3. _mpl-gallery
4. _mpl-gallery-nogrid
5. bmh
6. classic
7. dark_background
8. fast
9. fivethirtyeight
10. ggplot
11. grayscale
12. petroff10
13. seaborn-v0_8
14. seaborn-v0_8-bright
15. seaborn-v0_8-colorblind
16. seaborn-v0_8-dark
17. seaborn-v0_8-dark-palette
18. seaborn-v0_8-darkgrid
19. seaborn-v0_8-deep
20. seaborn-v0_8-muted
21. seaborn-v0_8-notebook
22. seaborn-v0_8-paper
23. seaborn-v0_8-pastel
24. seaborn-v0_8-poster
25. seaborn-v0_8-talk
26. seaborn-v0_8-ticks
27. seaborn-v0_8-white
28. seaborn-v0_8-whitegrid
29. tableau-colorblind10

★ MOST POPULAR STYLES:

- ✓ seaborn-v0_8
- ✓ ggplot
- ✓ dark_background
- ✓ bmh
- ✓ fivethirtyeight
- ✓ grayscale
- ✓ seaborn-v0_8-darkgrid
- ✓ seaborn-v0_8-whitegrid

HOW TO USE:

```
# Global (affects all plots)
plt.style.use('seaborn-v0_8')

# Temporary (only one plot)
with plt.style.context('dark_background'):
    plt.plot(x, y)

# Reset to default
```

```
plt.style.use('default')
```

5. Style Comparison - Side by Side

```
In [7]: print("\nSTYLE COMPARISON - SIDE BY SIDE")
print("="*70)

styles_to_compare = ['default', 'seaborn-v0_8', 'dark_background', 'bmh']
fig, axes = plt.subplots(2, 4, figsize=(20, 10))

for idx, style_name in enumerate(styles_to_compare):
    with plt.style.context(style_name):
        # Line plot
        ax1 = axes[0, idx]
        ax1.plot(x, y1, label='Sin', linewidth=2)
        ax1.plot(x, y2, label='Cos', linewidth=2)
        ax1.set_title(f'{style_name}\n(Line)', fontsize=11, fontweight='bold')
        ax1.legend()
        ax1.grid(True, alpha=0.3)

        # Bar plot
        ax2 = axes[1, idx]
        ax2.bar(categories, values)
        ax2.set_title(f'{style_name}\n(Bar)', fontsize=11, fontweight='bold')

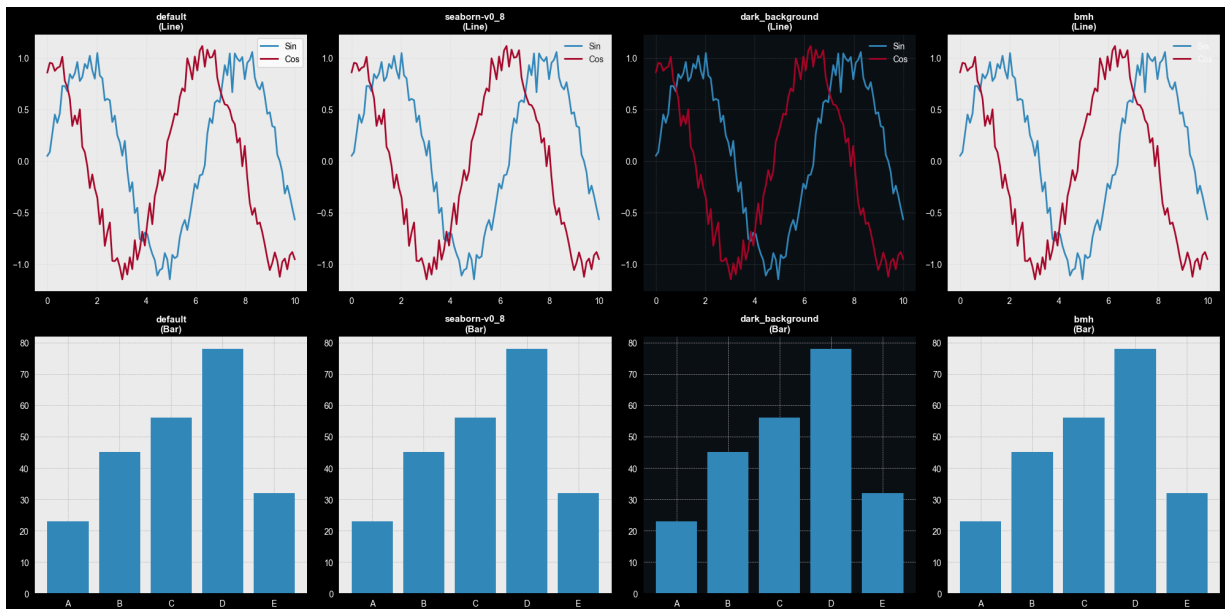
        # Match dark background
        if style_name == 'dark_background':
            ax1.set_facecolor('#0E1117')
            ax2.set_facecolor('#0E1117')

plt.tight_layout()
plt.show()

print("\n📊 COMPARISON SUMMARY:")
print("    DEFAULT      → Basic, dated, harsh colors")
print("    SEABORN       → Clean, modern, research-ready")
print("    DARK_BG       → Presentations, high-contrast")
print("    BMH           → Publication, formal, sophisticated")

print("\n🎯 DECISION TREE:")
print("    Research paper? → seaborn-v0_8 or bmh")
print("    Presentation? → dark_background")
print("    Notebook/analysis? → seaborn-v0_8")
print("    Dashboard? → dark_background or custom")
print("    Print/book? → bmh or grayscale")
print("="*70)
```

STYLE COMPARISON - SIDE BY SIDE



 **COMPARISON SUMMARY:**

- DEFAULT → Basic, dated, harsh colors
- SEABORN → Clean, modern, research-ready
- DARK_BG → Presentations, high-contrast
- BMH → Publication, formal, sophisticated

 **DECISION TREE:**

- Research paper? → seaborn-v0_8 or bmh
- Presentation? → dark_background
- Notebook/analysis? → seaborn-v0_8
- Dashboard? → dark_background or custom
- Print/book? → bmh or grayscale

6. Creating Custom Styles (Advanced)

```
In [8]: print("\nCREATING CUSTOM STYLES")
print("="*70)

# Method 1: Using rcParams dictionary
print("\n1 METHOD 1: rcParams Dictionary")
print("\nDefine custom style settings:")

custom_style = {
    'axes.facecolor': '#F0F0F0',
    'axes.edgecolor': '#333333',
    'axes.linewidth': 2,
    'axes.grid': True,
    'grid.color': 'white',
    'grid.linestyle': '-',
    'grid.linewidth': 1.5,
    'figure.facecolor': 'white',
    'font.size': 12,
    'axes.labelsize': 14,
    'axes.titlesize': 16,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
```

```

    'legend.fontsize': 11,
    'lines.linewidth': 2.5,
    'lines.markersize': 8,
    'axes.prop_cycle': plt.cycler('color', ['#E74C3C', '#3498DB', '#2ECC71',
}]

print("")
print("custom_style = {")
print("    'axes.facecolor': '#F0F0F0', # Light gray background")
print("    'axes.edgecolor': '#333333', # Dark gray axes")
print("    'axes.linewidth': 2,          # Thick axes")
print("    'axes.grid': True,            # Show grid")
print("    'grid.color': 'white',         # White gridlines")
print("    'lines.linewidth': 2.5,        # Thick lines")
print("    'axes.prop_cycle': cycler(...) # Custom colors")
print("    # ... more settings")
print("}")

# Apply custom style
with plt.rc_context(custom_style):
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(x, y1, label='Sin wave')
    ax.plot(x, y2, label='Cos wave')
    ax.plot(x, y3, label='SinxCos')
    ax.set_title('Custom Style Example', fontweight='bold')
    ax.set_xlabel('X axis')
    ax.set_ylabel('Y axis')
    ax.legend()
    plt.tight_layout()
    plt.show()

print("\n2 METHOD 2: Style File (.mplstyle)")
print("\nCreate a file 'my_style.mplstyle' with:")
print("""
# my_style.mplstyle
axes.facecolor: F0F0F0
axes.edgecolor: 333333
axes.linewidth: 2
axes.grid: True
grid.color: white
lines.linewidth: 2.5
font.size: 12
""")

print("\nThen use:")
print("    plt.style.use('path/to/my_style.mplstyle')")

print("\n3 METHOD 3: Combine Multiple Styles")
print("\nYou can layer styles:")
print("    plt.style.use(['seaborn-v0_8', 'dark_background'])")
print("    # Later style overrides earlier one")

print("\n💡 PRO TIPS:")
print("    • Save custom styles in ~/.matplotlib/stylelib/")
print("    • Name convention: company_name.mplstyle")
print("    • Test on multiple plot types")

```

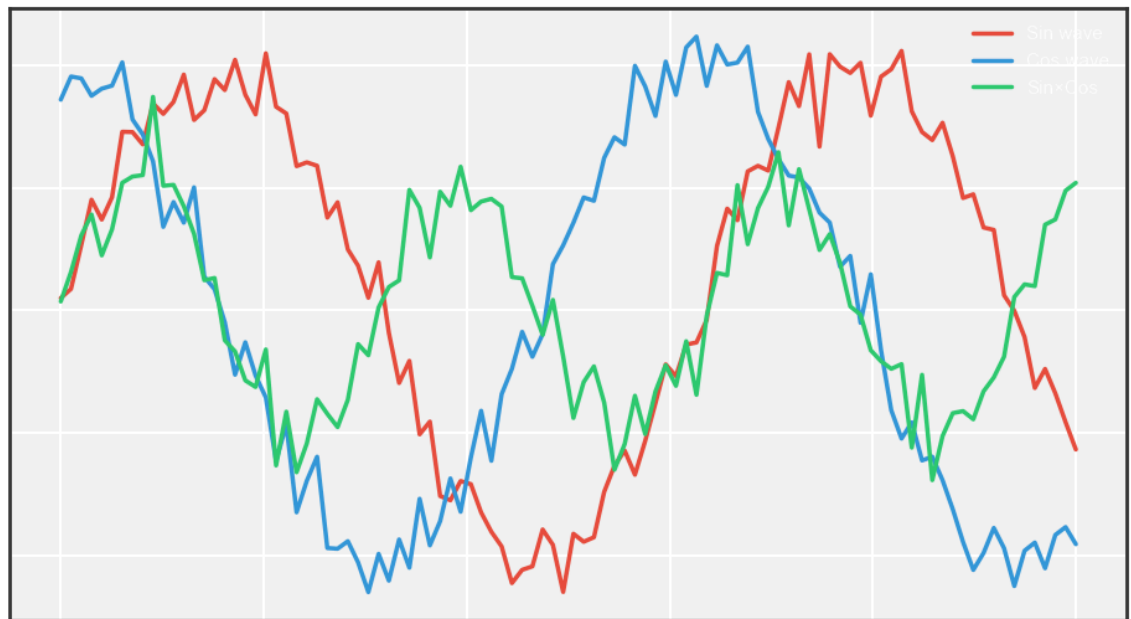
```
print("    • Check color accessibility (colorblind-friendly)")
print("=*70)
```

CREATING CUSTOM STYLES

1 METHOD 1: rcParams Dictionary

Define custom style settings:

```
custom_style = {
    'axes.facecolor': '#F0F0F0', # Light gray background
    'axes.edgecolor': '#333333', # Dark gray axes
    'axes.linewidth': 2,          # Thick axes
    'axes.grid': True,           # Show grid
    'grid.color': 'white',        # White gridlines
    'lines.linewidth': 2.5,       # Thick lines
    'axes.prop_cycle': cycler(...) # Custom colors
    # ... more settings
}
```



2 METHOD 2: Style File (.mplstyle)

Create a file 'my_style.mplstyle' with:

```
# my_style.mplstyle
axes.facecolor: F0F0F0
axes.edgecolor: 333333
axes.linewidth: 2
axes.grid: True
grid.color: white
lines.linewidth: 2.5
font.size: 12
```

Then use:

```
plt.style.use('path/to/my_style.mplstyle')
```

3 METHOD 3: Combine Multiple Styles

You can layer styles:

```
plt.style.use(['seaborn-v0_8', 'dark_background'])
# Later style overrides earlier one
```

💡 PRO TIPS:

- Save custom styles in ~/.matplotlib/stylelib/
- Name convention: company_name.mplstyle
- Test on multiple plot types
- Check color accessibility (colorblind-friendly)

=====

7. Quick Style Switching Workflow

```
In [9]: print("\nQUICK STYLE SWITCHING WORKFLOW")
print("=*70)

print("\n👛 REAL-WORLD WORKFLOW:")
print("""
# At top of notebook
import matplotlib.pyplot as plt

# Set style once
plt.style.use('seaborn-v0_8') # For analysis

# Do all your plotting...
plt.plot(x, y)
plt.show()

# Need dark plot for presentation?
with plt.style.context('dark_background'):
    plt.plot(x, y)
    plt.savefig('presentation_plot.png', facecolor='#0E1117')

# Back to seaborn automatically!
plt.plot(x, y) # Still seaborn style
""")
```

```

print("\n🎨 STYLE HELPER FUNCTION:")
print("")

def quick_plot(x, y, style='seaborn-v0_8', title='', save=False, filename='p
"""
    Quick plot with style selection

    Args:
        style: 'seaborn-v0_8', 'dark_background', 'bmh', etc.
        save: Save to file?
    """
    with plt.style.context(style):
        fig, ax = plt.subplots(figsize=(10, 6))
        ax.plot(x, y, linewidth=2.5)
        ax.set_title(title, fontsize=16, fontweight='bold')
        ax.grid(True, alpha=0.3)

        if style == 'dark_background':
            fig.patch.set_facecolor('#0E1117')
            ax.set_facecolor('#0E1117')

        if save:
            plt.savefig(filename, dpi=300, bbox_inches='tight',
                        facecolor=fig.get_facecolor())

    plt.show()

print("def quick_plot(x, y, style='seaborn-v0_8', save=False):")
print("    # Plots with chosen style and optional save")
print("    ...")
print("")
print("# Usage:")
print("quick_plot(x, y1, style='seaborn-v0_8', title='Analysis')")
print("quick_plot(x, y1, style='dark_background', title='Presentation', save

# Demo
print("\n📊 DEMO: Same data, different styles")
print("")
quick_plot(x, y1, style='seaborn-v0_8', title='Seaborn Style')
quick_plot(x, y1, style='bmh', title='BMH Style')

print("\n💡 PRODUCTION TIP:")
print("    Create a company style guide .mplstyle file")
print("    Share across team for consistent branding!")
print("=="*70)

```


QUICK STYLE SWITCHING WORKFLOW

=====

REAL-WORLD WORKFLOW:

```
# At top of notebook
import matplotlib.pyplot as plt

# Set style once
plt.style.use('seaborn-v0_8') # For analysis

# Do all your plotting...
plt.plot(x, y)
plt.show()

# Need dark plot for presentation?
with plt.style.context('dark_background'):
    plt.plot(x, y)
    plt.savefig('presentation_plot.png', facecolor='#0E1117')

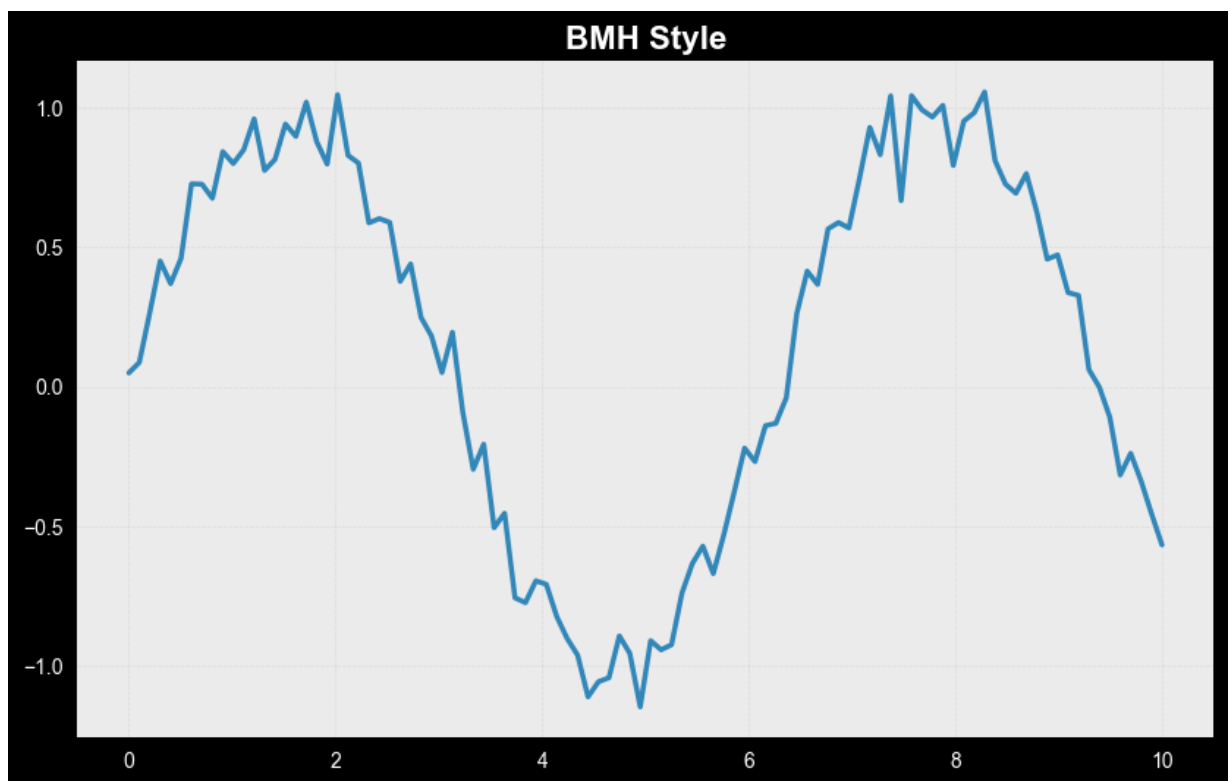
# Back to seaborn automatically!
plt.plot(x, y) # Still seaborn style
```

STYLE HELPER FUNCTION:

```
def quick_plot(x, y, style='seaborn-v0_8', save=False):
    # Plots with chosen style and optional save
    ...

# Usage:
quick_plot(x, y1, style='seaborn-v0_8', title='Analysis')
quick_plot(x, y1, style='dark_background', title='Presentation', save=True)
```

 DEMO: Same data, different styles



💡 PRODUCTION TIP:
Create a company style guide .mplstyle file
Share across team for consistent branding!

=====

Key Takeaways

```
In [10]: print("\n" + "="*70)
print("KEY TAKEAWAYS - MATPLOTLIB PLOTTING STYLES")
```

```
print("="*70)
```

```
takeaways = """
```

🌈 TOP 3 STYLES:

1. SEABORN-V0_8

- Clean, modern, muted colors
- Perfect for: Research, analysis, notebooks
- Code: `plt.style.use('seaborn-v0_8')`

2. DARK_BACKGROUND

- High contrast, vibrant colors on dark
- Perfect for: Presentations, dashboards, screens
- Code: `plt.style.use('dark_background')`

3. BMH (Bayesian Methods for Hackers)

- Sophisticated, publication-ready
- Perfect for: Papers, books, formal reports
- Code: `plt.style.use('bmh')`

🔧 ESSENTIAL COMMANDS:

```
# List all styles
plt.style.available
```

```
# Apply globally
plt.style.use('style_name')
```

```
# Temporary (context manager)
with plt.style.context('style_name'):
    plt.plot(x, y)
```

```
# Reset to default
plt.style.use('default')
```

```
# Custom style
plt.style.use('path/to/custom.mplstyle')
```

💡 PRO TIPS:

1. NEVER use default style in production
2. Use context manager for temporary style changes
3. Create custom `.mplstyle` for company branding
4. Dark backgrounds need HIGH-CONTRAST colors
5. Test plots on different screens/prints
6. Consider colorblind accessibility
7. Save with correct facecolor:
`plt.savefig('plot.png', facecolor=fig.get_facecolor())`

🎯 DECISION GUIDE:

Jupyter notebook? → `seaborn-v0_8`
Dark room presentation? → `dark_background`
Academic paper? → `bmh` or `seaborn-v0_8`
Dashboard? → `dark_background` or custom
Print/book? → `bmh` or `grayscale`

Website (light theme)? → seaborn-v0_8
Website (dark theme)? → dark_background

✗ COMMON MISTAKES:

- ✗ Using default style
- ✗ Not saving with facecolor on dark plots
- ✗ Low-contrast colors on dark backgrounds
- ✗ Forgetting to reset after context
- ✗ Not testing on target medium (screen vs print)



REMEMBER:

One line of code (`plt.style.use()`) can transform
amateur plots into professional visualizations!

.....

```
print(takeaways)
print("="*70)
```

=====

KEY TAKEAWAYS – MATPLOTLIB PLOTTING STYLES

=====

🎨 TOP 3 STYLES:

1. SEABORN-V0_8
 - Clean, modern, muted colors
 - Perfect for: Research, analysis, notebooks
 - Code: `plt.style.use('seaborn-v0_8')`
2. DARK_BACKGROUND
 - High contrast, vibrant colors on dark
 - Perfect for: Presentations, dashboards, screens
 - Code: `plt.style.use('dark_background')`
3. BMH (Bayesian Methods for Hackers)
 - Sophisticated, publication-ready
 - Perfect for: Papers, books, formal reports
 - Code: `plt.style.use('bmh')`

🔧 ESSENTIAL COMMANDS:

```
# List all styles
plt.style.available

# Apply globally
plt.style.use('style_name')

# Temporary (context manager)
with plt.style.context('style_name'):
    plt.plot(x, y)

# Reset to default
plt.style.use('default')

# Custom style
plt.style.use('path/to/custom.mplstyle')
```

💡 PRO TIPS:

1. NEVER use default style in production
2. Use context manager for temporary style changes
3. Create custom `.mplstyle` for company branding
4. Dark backgrounds need HIGH-CONTRAST colors
5. Test plots on different screens/prints
6. Consider colorblind accessibility
7. Save with correct facecolor:
`plt.savefig('plot.png', facecolor=fig.get_facecolor())`

🎯 DECISION GUIDE:

Jupyter notebook? → `seaborn-v0_8`
Dark room presentation? → `dark_background`
Academic paper? → `bmh` or `seaborn-v0_8`
Dashboard? → `dark_background` or `custom`

Print/book? → bmh or grayscale
Website (light theme)? → seaborn-v0_8
Website (dark theme)? → dark_background

✂ COMMON MISTAKES:

- ✗ Using default style
- ✗ Not saving with facecolor on dark plots
- ✗ Low-contrast colors on dark backgrounds
- ✗ Forgetting to reset after context
- ✗ Not testing on target medium (screen vs print)



REMEMBER:

One line of code (`plt.style.use()`) can transform
amateur plots into professional visualizations!

=====